Hashing

Hashing (creating a message digest) is one of the easiest operations in OpenSSL. A **hash** is a fixed-size value derived from your data that **cannot be reversed**. OpenSSL supports MD5, SHA-1, SHA-256, SHA-512, etc.

Here's a clear example using OpenSSL on Windows.

1 Hash a text file using SHA-256

Suppose you have a file `message.txt`:

```
Hello, this is a test message.
```

Command

```
openssl dgst -sha256 message.txt
```

Output

```
SHA256(message.txt)=
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
```

Explanation:

- `dgst` → digest command
- `-sha256` → SHA-256 algorithm
- Output is the **hash of the file**

2 Hash a file using SHA-512

```
openssl dgst -sha512 message.txt
```

Output

```
SHA512(message.txt)=
1f40fc92da241694750979ee6cf582f2d5d7d28e18335de05abc54d0560e0f530286dd02a2f1e
66f3b5a30f3
```

3 Hash a `.docx` file

```
openssl dgst -sha256 "document.docx"
```

Output Example

```
SHA256(document.docx)= 5d41402abc4b2a76b9719d911017c592
```

Explanation:

- Works the same way for **any file type**.
- The hash changes if **even one byte in the file changes**.

4 Create a hash and save it to a file

```
openssl dgst -sha256 "document.docx" > digest.txt
```

- The hash value is now saved in `digest.txt`
- You can compare it later to **verify file integrity**

---

5 Verify a hash

Suppose you have a hash file `digest.txt` with the correct SHA-256 hash of `document.docx`. You can check:

```
openssl dgst -sha256 -verify digest.txt "document.docx"
```

Note: Normally `-verify` is used with public keys for signatures. For simple hash comparison, just **recompute and compare manually**.

Summary Table

Operation	Command Example
SHA-256 hash of text file	`openssl dgst -sha256 message.txt`
SHA-512 hash of text file	`openssl dgst -sha512 message.txt`
SHA-256 hash of `.docx`	`openssl dgst -sha256 "document.docx"`
Save hash to file	`openssl dgst -sha256 "document.docx" > digest.txt`

Since we use same algorithm the digest message shows fixed-size value: Digest length is fixed, but Digest values differ with different input

Digest length is fixed

- Every hashing algorithm produces a **fixed-size output**.
- Examples:
  - **MD5** → 128 bits (16 bytes)
  - **SHA-1** → 160 bits (20 bytes)
  - **SHA-256** → 256 bits (32 bytes)
  - **SHA-512** → 512 bits (64 bytes)

No matter whether your input is 1 byte or 1 GB, the digest length **does not change**.

2. Digest values differ with different input

smalltext.txt - Notepad

File Edit Format View Help

Works the same way for any file type.
The hash changes if even one byte in the file changes.

message.txt - Notepad

File Edit Format View Help

Encrypting a file using 3DES
openssl enc -des-ede3-cbc -in plaintext.txt -out encrypted_3des.txt -pbkdf2 -iter 10000 -md sha256
Explanation:

- -des-ede3-cbc → Triple DES (EDE = Encrypt-Decrypt-Encrypt) in CBC mode
- -in plaintext.txt → original file
- -out encrypted_3des.txt → encrypted output
- -pbkdf2 -iter 10000 -md sha256 → secure key derivation from password

OpenSSL will prompt for a password. Remember it for decryption.

2Decrypting the 3DES file
openssl enc -d -des-ede3-cbc -in encrypted_3des.txt -out decrypted.txt -pbkdf2 -iter 10000 -md sha256
Explanation:

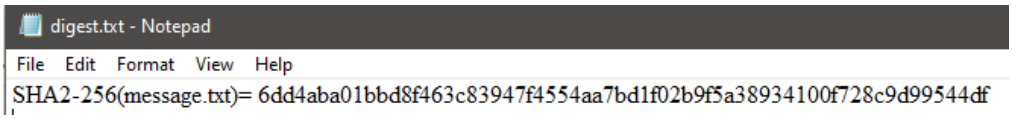
- -d → decrypt mode
- -des-ede3-cbc → same algorithm as encryption
- -in encrypted_3des.txt → encrypted file
- -out decrypted.txt → restored plaintext

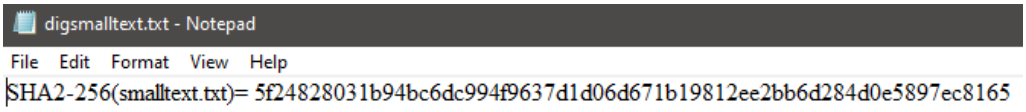
Notes

1. Password must match encryption password.
2. Triple DES uses 168-bit key internally.
3. On Windows OpenSSL 3.x, 3DES may require the legacy provider, just like DES. You may need:

-provider legacy -provider-path "C:\Program Files\OpenSSL-Win64\bin"

4. If the legacy provider cannot be loaded, use AES instead, which works everywhere





The **digest value (hash) depends entirely on the hashing algorithm**. Different algorithms produce **completely different hashes** even for the same input.

Example

Suppose the file `message.txt` contains:

```
Hello, world!
```

1. **SHA-256**

```
openssl dgst -sha256 message.txt
```

Output:

```
SHA256(message.txt)=
c0535e4be2b79ffd93291305436bf889314e4a3d80e24e22e1b3c7d5e6c1f4ab
```

2. **SHA-512**

```
openssl dgst -sha512 message.txt
```

Output:

```
SHA512(message.txt)=
309ecc489c12d6eb4cc40f50c902f2b4d0ed77ee511a7c7a9bcd3ca86d4cd86f989dd35bc5ff4
99670da34255b45b0cf
```

3. **MD5**

```
openssl dgst -md5 message.txt
```

Output:

```
MD5(message.txt)= fc3ff98e8c6a0d3087d515c0473f8677
```

Observations

- All hashes are **different**, even though the **input file is exactly the same**.
- The **length of the digest** also differs:
  - MD5 → 128 bits (16 bytes)
  - SHA-256 → 256 bits (32 bytes)
  - SHA-512 → 512 bits (64 bytes)
- Using a **different hashing algorithm changes the hash completely**, so you **cannot compare an MD5 hash with a SHA-256 hash**.

➡ **So, try the above using MD5 instead of sha256. And report the result**